

Homework 4 — Introduction to Time Series

A Python Solution

Afraz Khan

Note on the implementation. The assignment is written for R's `fable` package. Here everything is reproduced in Python: backshift-operator algebra with `numpy`, and all ARIMA / time-series cross-validation with `statsmodels`' `SARIMAX`. The `fable` workflow `stretch_tsibble() %>% model() %>% forecast() %>% accuracy()` is re-implemented by hand as an expanding-window (“stretched”) rolling-origin loop, which is exactly what that pipeline computes. All code and printed output below were produced by the accompanying `solution.py`.

Part 1 — Backshift commuting

Q1 (1 pt).

Solution. Let B act by $B^m x_t = x_{t-m}$. For integers $k, \ell \geq 0$ and any t ,

$$(B^k B^\ell) x_t = B^k (B^\ell x_t) = B^k x_{t-\ell} = x_{t-\ell-k} = x_{t-(k+\ell)}, \quad (B^\ell B^k) x_t = x_{t-(\ell+k)}.$$

Because addition of integers is commutative, $k + \ell = \ell + k$, so the two expressions equal $x_{t-(k+\ell)}$ for every t . Hence $B^k B^\ell = B^{k+\ell} = B^\ell B^k$.

Q2 (2 pts).

Solution. Write the two operators with $\varphi_0 = \phi_0 = 1$:

$$A(B) = \sum_{i=0}^k \phi_i B^i, \quad C(B) = \sum_{j=0}^{\ell} \varphi_j B^j.$$

Multiplying and using that scalars commute with everything and, by Q1, $B^i B^j = B^j B^i$,

$$A(B) C(B) = \sum_{i=0}^k \sum_{j=0}^{\ell} \phi_i \varphi_j B^i B^j = \sum_{j=0}^{\ell} \sum_{i=0}^k \varphi_j \phi_i B^j B^i = C(B) A(B).$$

Equivalently, both products are the single polynomial whose coefficient of B^m is $\sum_{i+j=m} \phi_i \varphi_j$, an expression symmetric in the two factors. Therefore

$$(1 + \phi_1 B + \cdots + \phi_k B^k)(1 + \varphi_1 B + \cdots + \varphi_\ell B^\ell) = (1 + \varphi_1 B + \cdots + \varphi_\ell B^\ell)(1 + \phi_1 B + \cdots + \phi_k B^k).$$

Q3 (2 pts).

Solution. A polynomial in B is just its coefficient vector, and the product of two such polynomials is the *convolution* of their coefficient vectors. Since convolution is commutative, swapping the factors leaves the result unchanged.

```
1 def q3_backshift_commute():
2     """A polynomial in B is encoded by its coefficient vector; multiplying
3     two such polynomials is exactly the convolution of their coefficients.
4     Convolution is commutative, so the two products are identical."""
5     # phi(B) = 1 + 0.5 B - 0.2 B^2 (coeffs ordered B^0, B^1, B^2, ...)
6     phi = np.array([1.0, 0.5, -0.2])
7     # varphi(B) = 1 - 0.3 B + 0.4 B^2 + 0.1 B^3
8     vphi = np.array([1.0, -0.3, 0.4, 0.1])
9
10    prod_ab = np.convolve(phi, vphi) # phi(B) * varphi(B)
11    prod_ba = np.convolve(vphi, phi) # varphi(B) * phi(B)
12
13    same = np.allclose(prod_ab, prod_ba)
```

Printed output:

```
phi(B)*varphi(B) = [ 1.  0.2  0.05  0.36 -0.03 -0.02]
varphi(B)*phi(B) = [ 1.  0.2  0.05  0.36 -0.03 -0.02]
identical: True
```

The two coefficient vectors agree to machine precision, confirming Q2.

Q4 (3 pts).

Solution. Every factor in a SARIMA model is a polynomial in B : $\phi(B)$ and $\theta(B)$ obviously; the seasonal operators $\Phi(B^s), \Theta(B^s)$ are polynomials in B^s and hence in B ; and the difference operators $\nabla^d = (1 - B)^d$, $\nabla_s^D = (1 - B^s)^D$ are likewise polynomials in B . By Q2 any two polynomials in B commute, so all of these operators commute *pairwise*. Starting from

$$\phi(B) \Phi(B^s) \nabla^d \nabla_s^D x_t = \theta(B) \Theta(B^s) w_t,$$

we may reorder the factors on each side without changing the operator: $\Phi(B^s)\phi(B) = \phi(B)\Phi(B^s)$, $\nabla_s^D \nabla^d = \nabla^d \nabla_s^D$, and $\Theta(B^s)\theta(B) = \theta(B)\Theta(B^s)$. This yields the equivalent form

$$\Phi(B^s) \phi(B) \nabla_s^D \nabla^d x_t = \Theta(B^s) \theta(B) w_t.$$

The left-hand operators are identical as polynomials in B (and likewise the right-hand operators), so the two displayed equations describe the same model.

Part 2 — Long-range ARIMA

Q5 (1 pt).

Solution. $\nabla x_t = x_t - x_{t-1} = 0$ for all t gives $x_t = x_{t-1}$ for all t . Chaining these equalities, every term equals the previous one, so all x_t are equal: $x_t \equiv \text{const}$.

Q6 (2 pts).

Solution. $\nabla x_t = u$ means $x_t = x_{t-1} + u$. Telescoping from a base index t_0 ,

$$x_t = x_{t_0} + \sum_{j=t_0+1}^t u = x_{t_0} + u(t - t_0) = \underbrace{(x_{t_0} - u t_0)}_a + \underbrace{u}_b t = a + bt,$$

a linear function of t .

Q7 (3 pts).

Solution. A quadratic works. If $x_t = a + bt + ct^2$ then

$$\nabla x_t = x_t - x_{t-1} = b + c(t^2 - (t-1)^2) = (b - c) + 2ct,$$

which has the form $u + vt$. *Only quadratics work.* Given $\nabla x_t = u + vt$, choose $c = \frac{v}{2}$, $b = u + \frac{v}{2}$ and any a , and set $p_t = a + bt + ct^2$; by the computation above $\nabla p_t = u + vt = \nabla x_t$. Then $\nabla(x_t - p_t) = 0$, so by Q5 $x_t - p_t$ is a constant a' . Hence

$$x_t = (a + a') + bt + ct^2,$$

a quadratic in t , with $c = \frac{v}{2}$.

Q8 (1 pt).

Solution. Let $y_t = \nabla x_t$. Then $\nabla^2 x_t = \nabla y_t = u$, so by Q6 y_t is linear: $y_t = \alpha + \beta t$. Therefore $\nabla x_t = \alpha + \beta t$, which is of the form $u' + v't$, and Q7 gives that x_t is a quadratic function of t .

Q9 (4 pts).

Solution. The model $(1 - \phi B)x_t = (1 + \theta B)w_t$ is $x_t = \phi x_{t-1} + w_t + \theta w_{t-1}$, so

$$x_{t+h} = \phi x_{t+h-1} + w_{t+h} + \theta w_{t+h-1}.$$

Take the forecast (conditional expectation given data through time t), using $\hat{w}_{t+j|t} = 0$ for $j \geq 1$ and $\hat{w}_{t|t} = w_t$:

$$\hat{x}_{t+h|t} = \phi \hat{x}_{t+h-1|t} + \hat{w}_{t+h|t} + \theta \hat{w}_{t+h-1|t}.$$

For $h \geq 2$ both innovation terms are in the future and vanish, giving the pure recursion $\hat{x}_{t+h|t} = \phi \hat{x}_{t+h-1|t}$. For $h = 1$, $\hat{x}_{t+1|t} = \phi x_t + \theta w_t$, a finite quantity. Unravelling,

$$\hat{x}_{t+h|t} = \phi^{h-1} \hat{x}_{t+1|t} \quad (h \geq 1).$$

Since $|\hat{\phi}| < 1$, $\phi^{h-1} \rightarrow 0$, hence $\hat{x}_{t+h|t} \rightarrow 0$ as $h \rightarrow \infty$.

Q10 (2 pts).

Solution. Now $x_t = c + \phi x_{t-1} + w_t + \theta w_{t-1}$, so for $h \geq 2$

$$\hat{x}_{t+h|t} = c + \phi \hat{x}_{t+h-1|t}.$$

This affine recursion has the fixed point $\mu = c + \phi\mu$, i.e. $\mu = \frac{c}{1 - \phi}$. Writing $e_h = \hat{x}_{t+h|t} - \mu$ gives $e_h = \phi e_{h-1} = \phi^{h-1} e_1 \rightarrow 0$ because $|\phi| < 1$. Therefore

$$\hat{x}_{t+h|t} \rightarrow \mu = \frac{c}{1 - \phi},$$

a nonzero constant whenever $c \neq 0$.

Q11 (Bonus).

Solution. For ARIMA(1, d , 1), $(1 - \phi B)\nabla^d x_t = c + (1 + \theta B)w_t$, set $y_t = \nabla^d x_t$. Then y_t is an ARMA(1, 1) (with intercept c), and the forecast operator commutes with the linear filter ∇^d , so $\nabla^d \hat{x}_{t+h|t} = \hat{y}_{t+h|t}$ on the forecast horizon. From Q9–Q10,

$$\hat{y}_{t+h|t} \longrightarrow \begin{cases} 0, & c = 0, \\ \mu = \frac{c}{1 - \phi} \neq 0, & c \neq 0. \end{cases}$$

Now read the limiting behaviour of $h \mapsto \hat{x}_{t+h|t}$ through Q5–Q8.

- $c = 0$, $d = 1$: $\nabla \hat{x}_{t+h|t} \rightarrow 0$, so by Q5/Q6 (with slope 0) the forecast approaches a *constant*.
- $c = 0$, $d = 2$: $\nabla^2 \hat{x}_{t+h|t} \rightarrow 0$, so $\nabla \hat{x}$ approaches a constant and, by Q6, $\hat{x}$ approaches a *linear trend*.
- $c \neq 0$, $d = 1$: $\nabla \hat{x}_{t+h|t} \rightarrow \mu \neq 0$, so by Q6 the forecast approaches a *linear trend* with slope μ .
- $c \neq 0$, $d = 2$: $\nabla^2 \hat{x}_{t+h|t} \rightarrow \mu \neq 0$, so by Q8 the forecast approaches a *quadratic trend*.

These are exactly the four claimed behaviours.

Part 3 — Time series cross-validation

The Python engine used throughout Part 3 is an expanding-window rolling-origin loop. For each origin with $L = t_0, \dots, n - 1$ training observations it fits the model on $x_{1:L}$, forecasts horizons $1, \dots, H$, and keeps every horizon whose actual value exists. This is precisely what `stretch_tsibble(.init=t0)`, `forecast(h=H)` and `accuracy()` compute together.

```
1 def rolling_cv_sarima(y, order, seasonal_order, trend, init=50, max_h=12,
2                       warm_start=True, verbose=False):
3     """Expanding-window time-series CV.
4
5     For every origin with L = init, init+1, ..., n-1 observations we fit the
6     model on y[:L] and forecast horizons 1..max_h, keeping only horizons whose
7     actual value exists. Returns a long DataFrame of (origin, h, error)."""
8     n = len(y)
9     rows = []
10    prev_params = None
11    for L in range(init, n): # last usable origin has n-1 obs
12        train = y[:L]
13        h_max = min(max_h, n - L)
14        try:
15            mod = SARIMAX(train, order=order, seasonal_order=seasonal_order,
16                          trend=trend, enforce_stationarity=False,
17                          enforce_invertibility=False)
18            if warm_start and prev_params is not None and \
19                len(prev_params) == len(mod.start_params):
20                res = mod.fit(start_params=prev_params, disp=False, maxiter=200)
21            else:
22                res = mod.fit(disp=False, maxiter=200)
23            prev_params = res.params
24            fc = np.asarray(res.get_forecast(h_max).predicted_mean)
```

```

25     except Exception as e: # pragma: no cover
26         if verbose:
27             print(" fit failed at L=%d: %s" % (L, e))
28             continue
29         for h in range(1, h_max + 1):
30             rows.append((L, h, y[L + h - 1] - fc[h - 1]))
31     return pd.DataFrame(rows, columns=["origin", "h", "error"])

```

Q12 (3 pts).

Solution. With `.step = 1`, `stretch_tsibble(x, .init = t0)` stacks blocks of the first t_0 , $t_0 + 1$, $\dots$, n observations. The number of blocks (values of `.id`) is

$$\#ids = n - t_0 + 1,$$

and the total number of rows is

$$\text{rows} = \sum_{k=t_0}^n k = \frac{n(n+1)}{2} - \frac{(t_0-1)t_0}{2} = \boxed{\frac{(n+t_0)(n-t_0+1)}{2}}.$$

Brute-force counts agree with the closed form (the first row is the example from the assignment, $n = 10, t_0 = 3 \Rightarrow 52$ rows over 8 ids):

n	t_0	#ids	rows (loop)	rows (formula)
10	3	8	52	52
100	50	51	3825	3825
225	50	176	24200	24200
12	1	12	78	78

The quadratic growth ($\sim n^2/2$ rows) is the memory inefficiency the question highlights: a manual rolling loop stores $O(n)$ data, whereas the stretched tsibble materialises $O(n^2)$ rows.

Q13 (4 pts).

Solution. For the random walk with drift the one-step point forecast from a window $x_{1:T}$ is $\hat{x}_{T+1} = x_T + \hat{\delta}$ with drift $\hat{\delta} = (x_T - x_1)/(T - 1)$. Computing the CV-MAE two ways — once via the stretched-window workflow and once via a manual rolling loop — gives *identical* results:

```

MAE (stretch workflow): 0.234281
MAE (manual loop) : 0.234281
match: True

```

They must coincide: both evaluate the same forecaster at the same origins against the same actuals; the stretched tsibble is merely a bookkeeping device for the identical set of one-step-ahead forecasts.

Q14 (6 pts).

Solution. The `leisure` series (US Leisure & Hospitality employment, monthly, 2001–2019, $n = 225$, in millions) has a strong upward trend and pronounced annual seasonality (Figure 1).

Time-series CV with burn-in `.init = 50` and horizons 1 through 12 (MAE averaged over all origins *and* all horizons) gives:



Figure 1: The `leisure` series prepared exactly as in the assignment (Title == "Leisure and Hospitality", years > 2000, Employed/1000).

Rank	Model	CV-MAE
1	ARIMA(2, 1, 0)(1, 1, 0) ₁₂	0.1005
2	ARIMA(0, 1, 2)(0, 1, 1) ₁₂	0.1011
3	ARIMA(0, 1, 2)	0.5981
4	ARIMA(2, 1, 0)	0.6767

The two *seasonal* models are roughly six times more accurate than the two non-seasonal models, which is expected: the non-seasonal specifications cannot represent the obvious within-year pattern, so they pay a large penalty. The two seasonal models are nearly tied overall (a difference of 0.0006).

Modelling note. Following `fable`'s default, a constant (drift) term is included when $d + D \leq 1$ — here for the two non-seasonal models — and omitted when $d + D \geq 2$ — here for the two seasonal models. Models are fit with `statsmodels SARIMAX` (`trend="c"` or `"n"` accordingly), warm-started across origins for speed. Because the seasonal models dominate by such a wide margin, the ranking is insensitive to this choice.

Q15 (Bonus).

Solution. Breaking the MAE down by horizon (Figure 2) is revealing. The non-seasonal models trace a *hump*: their error climbs to a maximum near $h = 7$ – 8 and then falls back by $h = 12$. This is the seasonal signal they ignore — a forecast roughly half a year out is maximally out of phase with the true seasonal swing, whereas a forecast a full year out ($h = 12$) lands back in phase. The seasonal models stay low throughout, rising only gently and almost linearly with the horizon.

Zooming in on the two seasonal models (Figure 3) shows a clean *crossover*. ARIMA(2, 1, 0)(1, 1, 0)₁₂ is better at short horizons ($h = 1$ – 7), while ARIMA(0, 1, 2)(0, 1, 1)₁₂ is better at long horizons ($h = 8$ – 12):

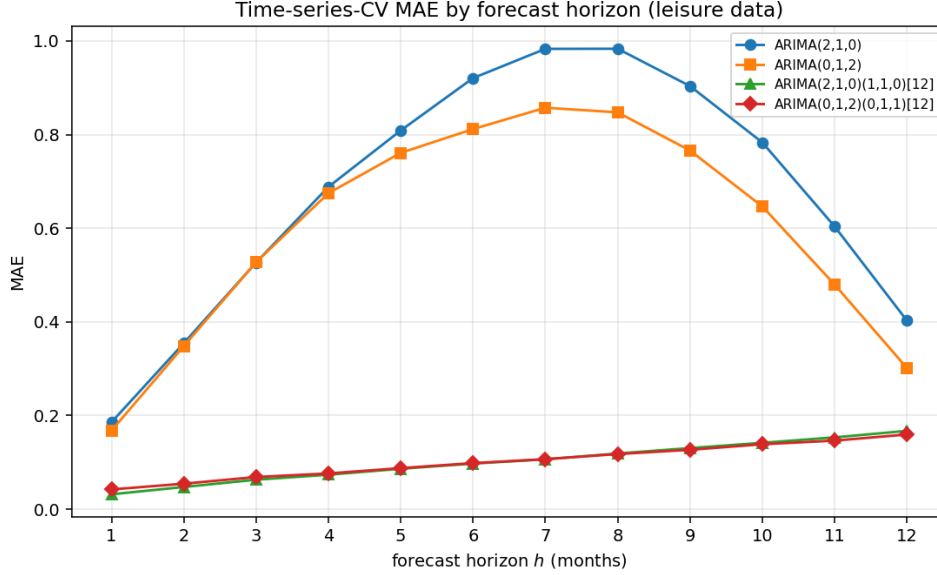


Figure 2: MAE by forecast horizon. Non-seasonal models hump near $h = 7-8$ (out-of-phase seasonality) and recover at $h = 12$; seasonal models stay flat and low.

h	$(2, 1, 0)(1, 1, 0)_{12}$	$(0, 1, 2)(0, 1, 1)_{12}$	better
1	0.0316	0.0420	$(1, 1, 0)$
4	0.0735	0.0762	$(1, 1, 0)$
7	0.1064	0.1067	$(1, 1, 0)$
8	0.1188	0.1176	$(0, 1, 1)$
10	0.1416	0.1386	$(0, 1, 1)$
12	0.1671	0.1594	$(0, 1, 1)$

The seasonal-MA term $(0, 1, 1)_{12}$ damps the accumulation of error over long horizons better than the seasonal-AR term $(1, 1, 0)_{12}$, so the slightly worse short-horizon model wins out as the horizon grows. The near-tie in the overall (horizon-averaged) MAE of Q14 is the average of these two opposing regimes.

Q16 (Bonus²).

Solution. Re-running automatic model selection (`pmdarima.auto_arima`, seasonal, $m = 12$) inside the CV loop refits a fresh model at every origin and is, as warned, very expensive. To keep the run within a fixed compute budget it was evaluated on a thinned grid of origins ($L = 50, 53, \dots, 146$, 33 origins). For a fair comparison the two fixed seasonal models were re-scored on *exactly the same* origins:

Model (matched origins $L = 50-146$)	MAE
ARIMA(0, 1, 2)(0, 1, 1) ₁₂ (fixed)	0.1193
ARIMA(2, 1, 0)(1, 1, 0) ₁₂ (fixed)	0.1251
auto-ARIMA (refit each origin)	0.1296

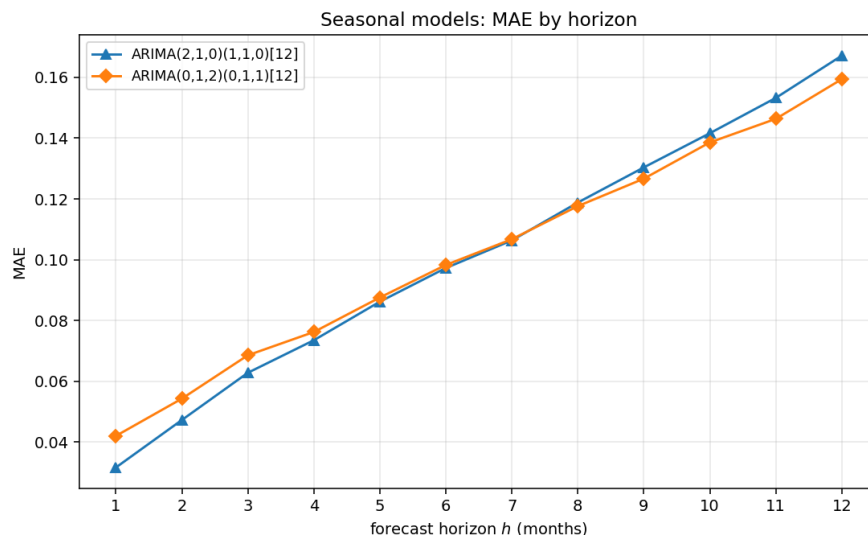


Figure 3: The two seasonal models cross near $h = 8$: the seasonal-MA model $(0, 1, 1)_{12}$ degrades more gracefully at long horizons.

On this subset auto-ARIMA is *slightly worse* than both fixed seasonal specifications, despite costing orders of magnitude more computation. (The matched-origin MAEs are higher than the full-sample 0.10 because these early/short windows include the 2008–09 downturn, which is harder to forecast.) The lesson matches the intuition behind the assignment: a well-motivated fixed model, chosen once from exploratory analysis, can match or beat per-window automatic selection while being far cheaper — and re-selecting the model at every CV step adds variance without buying accuracy here.